

THE COMPLETE HANDS-ON GUIDE

Ollama

Advanced Topics — Embeddings, RAG, Vision & Serving

The finale. Here you graduate from calling models to building real AI systems: semantic search with embeddings, retrieval-augmented generation over your own documents, talking to the raw REST API, working with images, and serving Ollama across a network.

PHASE 6 OF 6

Advanced Topics

Phase 6 — Advanced Topics

You can now install, run, script, converse with, stream from, and customise local models. This final phase combines all of it into the techniques behind modern AI applications. The centrepiece is **RAG** — Retrieval-Augmented Generation — which lets a model answer questions about *your* documents. We'll also go under the hood to the REST API, add vision, and put Ollama on the network.

WHAT YOU'LL LEARN IN THIS PHASE

Generating embeddings · cosine similarity · calling the REST API with `requests` and `httpx` · streaming newline-delimited JSON · building a complete RAG pipeline (with a note on ChromaDB) · multi-modal image models · and running Ollama as a network service.

6.1 Generating embeddings — `ollama.embed()`

6.2 Embedding vectors & cosine similarity

6.3 The REST API with `requests`

6.4 The REST API with `httpx` (async)

6.5 Streaming newline-delimited JSON

6.6 Building a RAG pipeline

6.7 Multi-modal models — passing images

6.8 Running Ollama as a network service

6.9 Hands-on project: a local RAG document Q&A

6.1 — Generating embeddings: `ollama.embed()`

An **embedding** is a list of numbers (a vector) that captures the *meaning* of a piece of text. Texts with similar meaning get similar vectors — even if they share no words. This is the foundation of semantic search and RAG. You generate them with a dedicated **embedding model** (recall the model categories from Phase 1), not a chat model.

```
import ollama

single = ollama.embed(
    model='embeddinggemma',
    input='The quick brown fox jumps over the lazy dog.',
)
print(len(single['embeddings'][0])) # vector length, e.g. 768
```

You can embed many texts at once by passing a list — far faster than looping, and essential when indexing lots of documents:

```
batch = ollama.embed(
    model='embeddinggemma',
    input=['First sentence', 'Second sentence', 'Third sentence'],
)
print(len(batch['embeddings'])) # 3 vectors
```

Recommended embedding model	Notes
<code>embeddinggemma</code>	Strong general-purpose default.
<code>qwen3-embedding</code>	High quality, multilingual.
<code>all-minilm</code>	Small and fast — great for low-resource machines.

EMBED() VS THE OLDER EMBEDDINGS()

The modern function is `ollama.embed(model=..., input=...)` — it supports batching and hits the `/api/embed` endpoint, which returns **L2-normalised (unit-length)** vectors. An older `ollama.embeddings(model=..., prompt=...)` exists for single inputs; prefer `embed()` for new code. **Golden rule:** use the *same* embedding model for both indexing your documents and embedding your queries.

6.2 — Embedding vectors and cosine similarity

To find which texts are most alike, you compare their vectors. The standard measure is **cosine similarity**: it measures the *angle* between two vectors, giving a score from -1 (opposite meaning) to 1 (identical meaning), ignoring their length. Higher means more similar.

```
import numpy as np

def cosine(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# embed two phrases and compare their meaning
v = ollama.embed(model='embeddinggemma',
                  input=['a happy dog', 'a joyful puppy', 'tax law'])['embeddings']
print(round(cosine(v[0], v[1]), 3)) # high — similar meaning
print(round(cosine(v[0], v[2]), 3)) # low — unrelated
```

A SHORTCUT FOR NORMALISED VECTORS

Because `/api/embed` returns unit-length vectors, their cosine similarity equals their simple dot product (the denominator is 1). So `np.dot(a, b)` alone is enough — a handy speed-up when ranking thousands of vectors.

6.3 — Calling the REST API directly with `requests`

The Python library is a convenience wrapper. Underneath, everything is plain HTTP to the server on `localhost:11434` (the same server from Phase 1). Sometimes you want to call it directly — from a language without an SDK, or to control the request precisely. The main endpoints are `/api/generate`, `/api/chat` and `/api/embed`.

```
import requests

resp = requests.post(
    'http://localhost:11434/api/generate',
    json={
        'model': 'gemma3',
        'prompt': 'Why is the sky blue?',
        'stream': False,      # get one complete JSON object back
    },
)
data = resp.json()
print(data['response'])      # the generated text
```

For chat, use `/api/chat` with the same `messages` list you know from Phase 3:

```
resp = requests.post(
    'http://localhost:11434/api/chat',
    json={
        'model': 'gemma3',
        'messages': [{'role': 'user', 'content': 'Hello!'}],
        'stream': False,
    },
)
print(resp.json()['message']['content'])
```

ONE KEY DIFFERENCE FROM THE SDK

Over the raw REST API, **streaming is ON by default**. If you want a single complete object, you must pass `"stream": false` — otherwise you'll receive a stream of partial JSON objects (see section 6.5).

6.4 — Calling the REST API with `httpx` (async-friendly)

`httpx` is a modern HTTP client with the same simple interface as `requests` but with first-class **async** support — pairing naturally with the async ideas from Phase 4. The synchronous form is nearly identical:

```
import httpx

r = httpx.post('http://localhost:11434/api/chat', json={
    'model': 'gemma3',
    'messages': [{'role': 'user', 'content': 'Hello!'}],
    'stream': False,
}, timeout=60)
print(r.json()['message']['content'])
```

And the async form, for concurrent calls inside web backends:

```
import asyncio, httpx

async def ask(prompt):
    async with httpx.AsyncClient(timeout=60) as client:
        r = await client.post('http://localhost:11434/api/generate',
            json={'model': 'gemma3', 'prompt': prompt, 'stream': False})
        return r.json()['response']

print(asyncio.run(ask('Say hi in French.')))
```

6.5 — Streaming newline-delimited JSON

When you stream over the REST API, the server doesn't send one big JSON document. It sends a sequence of small JSON objects, **one per line** — a format called NDJSON (newline-delimited JSON). To consume it, you read the response line by line and parse each line as its own JSON object. Each one looks just like a chunk from Phase 4.

```
import requests, json

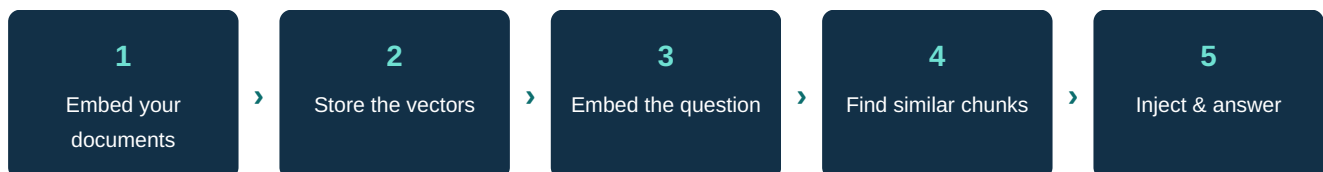
with requests.post(
    'http://localhost:11434/api/generate',
    json={'model': 'gemma3', 'prompt': 'Count to five.'}, # stream defaults to True
    stream=True,
) as resp:
    for line in resp.iter_lines():
        if not line:
            continue
        obj = json.loads(line) # parse one NDJSON object
        print(obj.get('response', ''), end='', flush=True)
        if obj.get('done'):
            break
```

THE PATTERN IN WORDS

Read each line → skip blanks → `json.loads` it → use its `response` (or `message.content` for `/api/chat`) → stop when `done` is true. This is exactly what the Python SDK does internally; now you could rebuild it yourself in any language.

6.6 — Building a RAG pipeline

Models don't know about your private documents, and they can't recall everything precisely. **Retrieval-Augmented Generation** solves this: instead of asking the model to know the answer, you *find* the relevant text yourself (using embeddings) and *inject* it into the prompt as context. The model then answers using facts you supplied. It's the single most important pattern for building trustworthy AI tools over your own data.



Each step maps onto something you already know: embedding (6.1), similarity (6.2), and chat with an injected context (Phase 3). The full working pipeline is the project at the end of this phase. The heart of it — retrieve, then answer — looks like this:

```
# given: `chunks` (your text pieces) and `chunk_vecs` (their embeddings)
def answer(question, chunks, chunk_vecs, k=3):
    q = ollama.embed(model='embeddinggemma', input=question)['embeddings'][0]
    # rank chunks by similarity to the question (dot product on unit vectors)
    ranked = sorted(zip(chunks, chunk_vecs),
                    key=lambda cv: -np.dot(q, cv[1]))
    context = '\n'.join(c for c, _ in ranked[:k])    # top-k chunks
    prompt = f'Use ONLY this context:\n{context}\n\nQuestion: {question}'
    return ollama.chat(model='gemma3',
                       messages=[{'role': 'user', 'content': prompt}]).message.content
```

IN-MEMORY VS A VECTOR DATABASE

For a few hundred chunks, a plain Python list plus cosine similarity (as above) is perfectly fast and dependency-free. For thousands or more, a purpose-built **vector database** like **ChromaDB** handles storage and fast search for you: `pip install chromadb`, then `collection.add(documents=..., embeddings=...)` and `collection.query(query_embeddings=..., n_results=3)`. Start in-memory; reach for ChromaDB when you outgrow it.

6.7 — Multi-modal models: passing images

Vision models (a category from Phase 1) accept images alongside text. With the Python library, add an `images` key to a message — a list of file paths or base64-encoded image data. The model can then describe pictures, read documents, or answer visual questions.

```
import ollama

response = ollama.chat(
    model='gemma3',          # a vision-capable model (e.g. gemma3, llava)
    messages=[{
        'role': 'user',
        'content': 'What is in this image?',
        'images': ['/path/to/photo.jpg'], # file path or base64
    }],
)
print(response.message.content)
```

From the command line it's even shorter — just include the path in your prompt (as you first saw in Phase 1):

```
ollama run gemma3 "What's in this image? /path/to/photo.jpg"
```

PAIRS BEAUTIFULLY WITH STRUCTURED OUTPUTS

Combine vision with the `format` schema trick from Phase 3 to get reliable, typed descriptions of images — e.g. a Pydantic model with `objects`, `colors` and `scene` fields, validated straight from the reply. Set `options={'temperature': 0}` for consistent results.

6.8 — Running Ollama as a network service

By default Ollama binds to `127.0.0.1` — reachable only from the same machine. To let other devices on your network use it (a home server, a shared GPU box), bind it to all interfaces by setting the `OLLAMA_HOST` environment variable.

```
# Expose Ollama to the whole local network (bind all interfaces)
OLLAMA_HOST=0.0.0.0:11434 ollama serve
```

Other machines then point at the server's IP address — including from Python via a custom client (Phase 2):

```
from ollama import Client
client = Client(host='http://192.168.1.50:11434') # the server's LAN IP
print(client.chat(model='gemma3',
    messages=[{'role': 'user', 'content': 'Hi from another
machine!'}]).message.content)
```

To make it permanent rather than a one-off, set the variable in the service configuration:

OS	How to set <code>OLLAMA_HOST</code> permanently
Linux (systemd)	<code>systemctl edit ollama.service</code> , add <code>Environment="OLLAMA_HOST=0.0.0.0:11434"</code> under <code>[Service]</code> , then <code>systemctl daemon-reload && systemctl restart ollama</code> .
macOS	<code>launchctl setenv OLLAMA_HOST "0.0.0.0:11434"</code> , then restart the Ollama app.
Windows	Quit Ollama, add <code>OLLAMA_HOST</code> in the user environment-variables settings, relaunch.

SECURITY & ACCESS NOTES

Exposing Ollama gives anyone who can reach that address full use of your models — only do this on trusted networks, and use a firewall or a reverse proxy (e.g. Nginx) for anything beyond your LAN. To allow browser extensions or other web origins, set `OLLAMA_ORIGINS`. Tunnels like ngrok or Cloudflare Tunnel can expose it securely off-network if needed.

Local RAG Document Q&A — chat with your own notes

The grand finale brings together embeddings, cosine similarity, retrieval and chat into one complete, runnable program — and it's fully local and private. Point it at some text, ask questions, and it answers using only your content. It uses an in-memory store (no external database) so it runs anywhere; a note shows the ChromaDB upgrade path.

Setup

```
pip install ollama numpy
ollama pull embeddinggemma      # embedding model (indexing + queries)
ollama pull gemma3              # chat model (answering)
```

The complete script — `rag_qa.py`

```
# rag_qa.py — a tiny, fully-local RAG question-answering system
import ollama
import numpy as np

MBED_MODEL = 'embeddinggemma'
CHAT_MODEL = 'gemma3'

# --- your knowledge base: split into small chunks (one fact each) ---
DOCUMENTS = [
    'Ollama runs large language models locally on your own machine.',
    'The Ollama server listens on port 11434 by default.',
    'Use ollama pull to download a model and ollama rm to delete it.',
    'A Modelfile bakes a system prompt and parameters into a custom model.',
    'Embeddings turn text into vectors for semantic search and RAG.',
]

def build_index(docs):
    """Step 1 + 2: embed every document and keep the vectors in memory."""
    vecs = ollama.embed(model=MBED_MODEL, input=docs)['embeddings']
    return [np.array(v) for v in vecs]

def retrieve(question, docs, vecs, k=3):
    """Step 3 + 4: embed the query, rank docs by cosine similarity."""
    q = np.array(ollama.embed(model=MBED_MODEL, input=question)['embeddings']
    [0])
    # /api/embed returns unit vectors, so dot product == cosine similarity
    scored = sorted(zip(docs, vecs), key=lambda dv: -float(np.dot(q, dv[1])))
    return [doc for doc, _ in scored[:k]]
```

```

def answer(question, context_chunks):
    """Step 5: inject the retrieved context and let the model answer."""
    context = '\n- '.join(context_chunks)
    prompt = (
        f'Answer the question using ONLY the context below. '
        f'If the answer is not present, say you do not know.\n\n'
        f'Context:\n- {context}\n\nQuestion: {question}'
    )
    r = ollama.chat(model=CHAT_MODEL,
                    messages=[{'role': 'user', 'content': prompt}])
    return r.message.content

def main():
    print('Indexing documents...')
    vecs = build_index(DOCUMENTS)
    print('Ready. Ask a question (or "quit").\n')

    while True:
        question = input('Q: ').strip()
        if question.lower() in {'quit', 'exit', ''}:
            break
        chunks = retrieve(question, DOCUMENTS, vecs)
        print(f'\n[retrieved {len(chunks)} chunks]')
        print('A:', answer(question, chunks), '\n')

if __name__ == '__main__':
    main()

```

Run it

```

python rag_qa.py
Q: What port does Ollama use?
A: Ollama's server listens on port 11434 by default.

```

SCALING UP WITH CHROMADB

To swap the in-memory store for a real vector DB: `pip install chromadb`, create a collection, `collection.add(ids=..., documents=DOCUMENTS, embeddings=vecs)` once, then replace `retrieve()` with `collection.query(query_embeddings=[q], n_results=3)`. The rest of the pipeline is unchanged — proof that the pattern, not the storage, is what matters.

EXTEND IT YOURSELF

- (1) Load real text from files and split into overlapping chunks. (2) Stream the answer with `stream=True` (Phase 4). (3) Show the source chunk(s) behind each answer so users can verify. (4) Add a vision step that answers questions about an image using section 6.7.

What you achieved

You generated embeddings, ranked text by cosine similarity, and built a complete retrieval-augmented generation system that answers questions about your own documents — entirely offline. Along the way you learned to call the REST API directly, stream NDJSON, work with images, and serve Ollama across a network. You can now build real, private AI applications end to end.

You've completed the Complete Ollama Guide

Across six phases you went from zero to building production-shaped AI systems, entirely on your own hardware:

Phase	What you can now do
1 • Setup	Install Ollama and manage models from the CLI.
2 • Python	Drive models programmatically and read responses.
3 • Prompting	Design conversations, memory and structured output.
4 • Streaming & Async	Build responsive, concurrent AI tools.
5 • Modelfile	Create and share your own custom models.
6 • Advanced	Embeddings, RAG, the REST API, vision and serving.

Where to go next: explore tool calling (letting models call your functions), the "thinking" capability for reasoning models, and the official docs at docs.ollama.com for the full API reference. Everything you build from here rests on the foundation in these six phases. Happy building.